The Professional Java Style Guide

Haim Michael

Cover photo by Negative Space:
https://www.pexels.com/photo/pink-white-black-purple-blue-textile-web-scripts-97077.

This book is based on my personal experience and on Oracle's Java Code Conventions document that was published at https://www.oracle.com/java/technologies/javase/codeconventions-contents.html.

Table of Contents

Introduction

This book presents a professional approach to writing Java code that is readable, maintainable, and aligned with long-term system evolution. This book presents a set of conventions grounded in real-world experience. These are patterns that reduce ambiguity, prevent defects, and enable teams to collaborate effectively.

Style guides do not appear by accident. They usually emerge after years, and in some cases, even decades of real-world experience gained by highly skilled developers working on large, long-lived systems.

There are already many excellent resources on Java style and best practices, including official Java documentation and well-known books such as Effective Java by Joshua Bloch. These resources are comprehensive and authoritative, but they can also be overwhelming for those taking their first steps. In addition, there are several best practices that these resources still don't cover. This Java Style Guide does not compete with or replace those works. This book is a short, focused, and approachable guide that helps beginners and students concentrate on the most important style guidelines first. The ones that have the greatest impact on readability, consistency, and long-term maintainability.

Rather than covering every possible rule or edge case, this book focuses on the guidelines I consider most important for us all to follow. The goal is to help readers develop good habits early: writing Java code that is easy to read, review, and evolve. This Java Style Guide is ideal for students, beginners, and early-career Java developers who want a clear starting point for writing Java with style without getting lost in hundreds of pages of rules and theory.

Comments

The comments serve as the communication layer between developers, reviewers, and tools. The comments must provide additional information that is not immediately obvious from the code. Having redundant comments might create noise and degrade readability.

Javadoc Comments

The Javadoc comments should be added to every public and every protected element from the following list:

- Classes
- Enums
- Interfaces
- Exceptions
- Methods
- Fields

When dealing with fields, we will add the Javadoc comments to every field, excluding the local ones.

When dealing with private and package-friendly elements, we will usually avoid adding the Javadoc comments, except for specific cases in which we would like to expose the existence of these members and explain them because of their unique role or their unique behavior.

The Javadoc comment we add will eventually be reflected in the API documentation we will create for our code. The API documentation is meant to serve those who don't have access to the code or don't want to access it to understand it. We will create the API documentation using the javadoc utility.

You can find a detailed guide by Oracle for writing Javadoc comments at https://www.oracle.com/il-en/technical-resources/articles/java/javadoc-tool.html.

You can find a detailed guide by Oracle for using the javadoc utility at https://docs.oracle.com/en/java/javase/25/javadoc/javadoc-guide.pdf. The Javadoc comments should describe intent, contract, and usage. They should avoid explaining the obvious, and they should avoid explaining the implementation unless that information is important to those who use the class.

```
/**
 * Calculates the total price incl tax
 *
 * @param basePrice the original price
 * @param taxRate the tax rate to apply
 * @return the total price incl tax
 * @throws IllegalArgumentException
 * if basePrice is negative
 */
public double calculateTotal(
        double basePrice, double taxRate) {
    ...
}
```

Based on my personal experience, it is highly useful to learn from the API comments we can find inside the source code of the various Java classes the Java SDK includes. You can easily find that source code at https://github.com/openjdk/jdk/blob/master/src.

Block Comments (C Style)

The block comments (/* … */) can be added at the beginning of the method, or the class, or the file, to explain non-trivial algorithms, design decisions, or a specific data structure we were using. These comments are highly relevant when dealing with complex code that its understanding is not obvious.

```
public void processData() {
/*
This method processes the dataset in
two phases:
1. Filtering invalid entries
2. Aggregating results using rolling
*/
    ...
}
```

Inline Comments (C++ Style)

The single-line comments (//) should be used to describe specific logical sections of the code. Usually, we will add them before the logical section (on top of it).

We won't add them to the end of a line of code. Doing so will make it very uncomfortable to read them when reviewing the code.

These single-line comments help with structuring the code into meaningful blocks, and they usually improve the readability.

```
// Create GUI components
JButton bt = new JButton("OK");
JTextField tf = new JTextField(10);

// Register event listeners
button.addActionListener(...);
```

We should avoid adding comments to obvious statements, as in the following example.

```
// increment i
i++;
```

Based on my experience over the years, we should prefer clear code over excessively long comments. Exaggerating with the comments creates noise that eventually damages the entire development process.

The comments should explain the code and provide answers for questions such as why the code does what it does. Explaining what the code does might be redundant.

Keeping the comment concise and up to date is the best approach. Creating the comments together with the code will usually keep them concise and accurate.

Whenever the code changes, we should update the comments accordingly. Based on my experience, doing it together with the changes we introduce in the code is the best approach.

Well-written comments complement well-written code. When both are aligned, the software system we develop becomes significantly simpler to understand, maintain, and evolve.

Identifiers

The identifiers are the names that we, the developers, choose to give to the variables, functions, and classes we define. When choosing good names, the cognitive load is reduced. The code becomes simpler to understand. The maintainability improves.

The identifiers in our code can be categorized into the following categories:

- Class names
- Interface names
- Enum names
- Method names
- Variable names
- Constant names
- Package names

Each category has its specific conventions that we should follow. In addition, there are general naming principles that apply to all categories. The names must be meaningful and descriptive. We should avoid abbreviations unless the abbreviations we intend to use are widely understood. It is highly important that the names we pick reflect the intent in order to reduce the cognitive burden. We'd better avoid misleading or ambiguous names.

Classes, Interfaces, Enums, Exceptions, and Records

When naming identifiers in this category, we should use PascalCase (UpperCamelCase). The name should start with a capital letter, and the rest of the letters should be in lowercase. When the name includes more than one word, each and every word will start with a capital letter. This way, when reading the name, the words it includes will stand out.

The name we pick for a class or a record must be a noun, and it shouldn't be in plural form (e.g. `Student` would be a great pick. `Students`, on the other hand, won't be a good pick).

When naming an interface, we should aim for a name that describes a capability or role (e.g. `Transfrable` is a great name. `Box` is not).

The name we pick for an enum should describe a set of constants (e.g., `Season` can be a great name for an enum that includes the names of the four seasons, `AUTUMN`, `WINTER`, `SPRING`, and `SUMMER`). The names we pick for the enum's possible values should include capital letters only.

The names we pick for exception classes should end with the word `Exception`.

Methods

When naming a method, we should use camelCase (lowerCamelCase).

```
public void calculateTotalIncome() { … }
```

The name we pick for a method should be a verb or include one. The name should clearly describe the action performed.

```
public void printReport() { … }
```

When defining the getters and setters, the name of the method will start with either `get` or `set`, and the word that will come right after will start with a capital letter.

```
public double getWidth() {
..
}

public void setWidth(double value) {

..
}
```

When defining a method that returns a boolean, its name should start with one of the following words: `is`, `has`, `can`, or `should`. If the use of the word `is` fits the case, that should be the preferred option.

```
public boolean isDaemon() {
…
}
```

Variables

When naming a variable, we should use camelCase. The name should use lowercase letters, and if it includes more than one word, the second word onward should start with a capital letter.

```
privat int numOfStudents = 10;
```

The names of all variables should be descriptive nouns. We'd better avoid single-letter names, except for very specific cases, such as loops.

```
for(int i=0; i<10; i++) {
  System.out.println("i="+i);
}
```

Constants

You should use UPPER_CASE with underscores. In most cases, the constants should be `static final`. Having a constant defined as an instance variable is technically possible. However, doing so might make sense only if each and every instance has a different value assigned to that constant. In most cases, the constant will have the same value for all objects. Therefore, in most cases, we will prefer to define the constant as a static final variable.

```
public static final int MAX_SPEED = 120;
public static final String ENCODING = "UTF-8";
```

Packages

We should name our packages with lowercase letters only, with a dot-separated structure. The name should reflect a hierarchy that starts with the domain name of the organization that develops the package.

```
package com.lifemichael.samples;
```

We should avoid upper cases and underscores.

Type Consistency and Semantic Alignment

The names we pick should align with the type and the behavior of the identifier. When naming a collection, we should use the name in its plural form.

```
List<Book> books;
```

~~`List<Book> book;`~~ `//poor`

When naming a variable that holds a single object, that name should be singular.

```
Invoice invoice;
```

~~`Invoice invoices;`~~ `//poor`

Avoiding Noise and Redundancy

You shouldn't repeat context that is already provided by the type of the scope.

```
User user;
```

~~`User userObject;`~~ `//poor`

You should avoid encoding the type in the names you choose. Having the type mentioned when the variable is declared is sufficient for every human to understand that the variable we create will hold a value of that specific type.

String userName;

~~`String userNameString;`~~ `//poor`

Boolean Naming

Variables of boolean type and methods that return boolean values should be named as conditions.

```
boolean isActive;

boolean hasAccess;

boolean canExecute;
```

We should avoid vague names.

```
boolean flag; //bad

boolean isTurboEnabled;
```

When naming the identifiers, we should always prefer having consistency over our personal preferences. Consistency contributes to cleaner, simpler code that is easier to understand. I learned that the hard way. I hope you take my advice on that.

Make sure the names follow the same naming patterns throughout the project. Follow the established conventions within the team or organization where you work. Try to avoid mixing different styles.

Well-chosen identifiers act as documentation, making it simpler for everyone to understand the code. The need for detailed comments decreases, and the structure of the system becomes self-explanatory.

Classes

The classes are the primary building blocks of the object-oriented design in Java. Their structure and consistency directly affect maintainability, correctness, and extensibility.

Constructors and Initialization

Every class you define should include a definition of the primary constructor that performs the full initialization of the newly created object. The other constructors in that class should delegate the initialization to the primary constructor. That will help us avoid code duplication and ensure consistent initialization logic.

```
class Rectangle {

	private int width;
	private int height;

	Rectangle(int width, int height){

		...

	}
	Rectangle() {

		this(10,10);

	}

}
```

Based on my experience, taking this approach will prevent divergence between constructors.

Validation

The validation logic should be kept inside the setter methods. The constructors should invoke the setters rather than assigning the values directly. This way, we will avoid duplication of the validation logic and ensure that all assignments pass through a single validation layer.

```
public void setAge(int age) {
    if (age < 0) {
```

```
        throw new Exception(...);
    }
    this.age = age;
}
```

This guarantees that validation rules are consistently enforced regardless of how the values are assigned.

Structure

Maintaining a consistent internal structure aligned with the conventions the Java API follows will improve the clarity of the class definition. Make sure you define the class in according with the following order:

1. Fields
2. Constructors
3. Public Methods
4. Protected Methods
5. Private Methods
6. Nested Classes

The `equals` and `hashCode` Methods

Whenever equals is overridden, hashCode must also be overridden. Both methods must be logically consistent. When two objects are equal, then calling hashCode on each one of them should return the same value, and vice versa. Failing to maintain this rule might lead to incorrect behavior when using collections, such as `HashSet` and `HashMap`.

The `toString` Method

Whenever you define a new class, make sure it includes the definition of the toString method. This method provides a human-readable representation of the object. Based on my experience, having that method already defined in a class helps with debugging and with creating the log messages.

`toString` is strongly recommended. It provides a human-readable representation of the object, which is essential for debugging and logging.

```
@Override
public String toString() {
    return "User{name='" + name + "', age=" + age +
"}";
}
```

Access Modifiers

The fields you include in the class definition should be private unless there is a strong, well-justified reason otherwise. This approach enforces encapsulation and prevents unintended external modifications. When needed, the access should be controlled via getters and setters.

Method Arguments Validation

Every method should start with validating the values (arguments). When a value doesn't pass the validation, the method should throw an exception.

```
public void updateEmail(String email) {
  if (email == null || email.isEmpty()) {
    throw new IllegalArgumentException("...");
  ...
}
```

Overriding Methods

Whenever overriding a method, the `@Override` annotation must be added to the new method's definition. This way, the compiler will be able to verify that we indeed have overriding and improve the readability. If we try to compile code that includes the `@Override` annotation but there is no overriding, you will get a compilation error.

```
@Override
public String toString() {
```

```
    ...
}
```

Well-structured classes reduce duplication, enforce consistency, and make behavior explicit. When these practices are applied systematically, the codebase becomes easier to reason about and safer to evolve.

Interfaces

The interfaces define contracts that describe what a component does without dictating how it does it. The use of interfaces can lead to flexible, decoupled, and testable systems.

Separate Interface from Implementation

The separation between the definition of an interface and the classes that implement it provides a clear separation. The interfaces define the behavior, and the classes provide the implementation.

```
public interface Flyable {
  public abstract void fly();
}

public class Bird implements Flyable {
  …
  public void fly() {
    …
  }
}
```

This clear separation allows multiple implementations and reduces coupling between the components. This separation also assists with writing tests. Having an interface makes it simpler to develop mocks and generate stubs.

Using The Interface Type

Whenever you have a variable that holds the reference to an object, that was created from a class that implements an interface, prefer, when possible, to set the type of that variable to be that interface.

```
Flyable flyable = new Bird(...);
```

This separation will allow us the flexibility to assign the variable with a reference to any object, as long as that object was created from a class that implements the interface we are dealing with.

```
List<Product> list = new LinkedList<Product>();
List<Product> list = new ArrayList<Product>();
```

Whenever we can use the interface as a type, we should. Whether dealing with a local variable, an instance variable, a static variable, or even the type of the returned value by a specific function, we'd better do so. That will improve the flexibility for changes in our program.

Prefer Interface over Abstract Classes

I have taught Java programming for nearly 30 years. During the years, I have noticed that many students are confused when they are asked about the difference between an interface and an abstract class. After all, we cannot instantiate an interface, and we also cannot instantiate an abstract class.

Nevertheless, although this similarity, there are several fundamental differences. The abstract class can include the definition of instance variables and the definition of a constructor. The interface cannot. The interface can be implemented in a class that implements more interfaces and extends a specific other class. The abstract class can be extended by a class that implements its abstract methods.

The class that implements the interface can implement more interfaces and can even extend another class. The class that extends an abstract class cannot extend another class. Multiple inheritance is not allowed in Java.

Therefore, when the type of a variable is an interface, we can assign that variable a reference to any object as long as it was created from a class that implements that interface. When the type of a variable is a class or an abstract class, we are limited to assigning that variable a reference to an object instantiated from a class that extends the type of that variable. Having a class that is forced to extend a specific class means that it won't be able to extend another class.

The bottom line is that we enjoy more flexibility when the variable's type is an interface than when it is an abstract class.

The use of abstract classes is appropriate only when there is a shared state.

My recommendation is to stick with the SRP (Single Responsibility Principle) and keep the interfaces focused and minimal. Having to

implement an interface that forces us to implement irrelevant methods will violate the SRP principle.

```
public interface Readable {

  public abstract String read();

}
```

The use of interfaces is one of the most important concepts of professional software development. The support for interfaces exists in many programming languages. Java is just one of them. When used correctly, they enable systems that are easier to extend, test, and evolve without introducing unnecessary coupling.

Exceptions

Exception handling defines how the system responds to a failure. The exception handling should be explicit, consistent, and domain-oriented.

Avoid Catching Generic Exceptions

Handling exceptions using a catch block that refers to the `Exception` type, or even worse, the `Throwable` type, will result in having a catch block that will execute for any exception type that extends these two. That might lead to code that handles specific exception types incorrectly. Therefore, don't use catch blocks that handle Exception (or worse, Throwable) unless there is a very strong justification for that.

```
try {
  …
  …
} catch(Exception e) {  //poor

  …

}
```

Instead, catch specific exception types that you can meaningfully handle.

Use Project Exception Types

Define a custom exception type (or even types) for the project you are working on, and use it consistently throughout the project. When creating a custom type, you can optimize it for your project's needs by defining it with the right members (fields and methods) for describing exceptions in your specific project.

```
public class AccountingException
  extends Exception {
    public AccountingException(
      String msg) {

        super(msg);
```

```
    }
    public AccountingException(
      String msg,
      Throwable cause) {

        Super(msg, cause);

    }
}
```

Whenever an exception takes place, wrap it in your project-specific exception. That will provide a consistent error model.

```
try {
    ...
} catch (SQLException e) {
    throw new AccountingException("...", e);
}
```

Interfaces Methods Signature

Include in your project's interfaces' methods' signatures the exception types you defined specifically for your project. That will allow classes that implement these interfaces to handle low-level exceptions and instead throw one of the project's specific exception types.

```
public interface ProductsDAO {
  public Product getProduct(int id)
    throws DAOException;
  public Product[] getProducts
    throws DAOException;
}
```

Group Atomic Operations in Single Try-Catch

When having a block of code that represents a single logical operation, and failure invalidates the entire operation, you should wrap the whole block in one try-catch statement.

```
try {
   // step 1 …
   // step 2 …
   // step 3 …
} catch(DAOException e) {
  // …
}
```

When something goes wrong, we should stop the execution of the entire operation. There is no point in handling step 1, that failed, and then trying to execute step 2, knowing that without a successful completion of step 1, step 2 will fail as well.

Fragmenting such logic into multiple try-catch blocks makes sense when a partial recovery is meaningful only.

Runtime Exceptions

The runtime exceptions are described using classes that extend `RuntimeException` directly or indirectly. When one of these classes is instantiated, the object we get represents a failure, which, in most cases, could be avoided by writing better code.

The `NullPointerException` type, for instance, is instantiated in order to describe a scenario in which a method was invoked using a variable that holds null. Such a scenario shouldn't happen. The code should be fixed. Handling such an exception using try and catch means that something that should have happened and failed will never happen.

The `ArrayIndexOutOfBoundsException` type, for instance, is instantiated whenever code tries to access an array using an index that violates the array size. Code that violates the array size should be fixed.

There are still rare cases in which we might prefer to handle exceptions of the `RuntimeException` type. I well remember when my company developed a game for the Nokia 7650, and the client told us that on his end, he saw a `NullPointerException` message on the screen. It took us some time to understand what happened. Eventually, we understood that the client had a Nokia 7650 with a new firmware that had a small bug whenever code in Java tried to play sound. In most cases, the method that played the sound worked fine. From time to time, it threw a `NullPointer`Exception. The solution was to wrap the code that called that method in a try-catch block. The result was a game in which, from time to time, the sound of the cannon wasn't heard. On the other hand, at least we didn't have a `NullPointerException` message on the screen.

Exceptions Class Design

When defining a custom exception type, make sure your definition includes (at a minimum) a constructor that takes the message that describes what happened, and a constructor that takes both the message and the low-level exception that was caught, and now, a new exception of the type you define will be thrown instead.

```
public class DAOException extends Exception{

  public DAOException(String msg,
                      Throwable cause) {
    ...
  }

}
```

The second constructor will actually allow the code that catches the DAOException to retrieve information about the low-level exception.

About The Author

Haim Michael is the Founder and CEO of Zindell Technologies, a company focused on advancing software development in the AI era. With more than 20 years of hands-on experience, he operates at the intersection of software engineering, architecture, and artificial intelligence, helping organizations rethink how modern systems are designed and built.

Haim has led initiatives across enterprise environments, academia, and global developer communities, bringing a pragmatic yet forward-looking approach to technology adoption. His work emphasizes structured, specification-driven development as a foundation for building reliable, scalable, and AI-integrated systems.

In parallel to his executive role, Haim has delivered training and strategic guidance to leading technology companies and has lectured at top academic institutions, including Bar-Ilan University, HIT, Shenkar, and the Technion. He is recognized for staying at the forefront of software development practices, continuously translating emerging capabilities, particularly in AI, into actionable engineering and business value.